Introduction

Programming: Everyday Decision-Making Algorithms

LECTURER

Dr. Nils Roemer, External Lecturer

AFFILIATION

Planningio GmbH

Today in one sentence

In this course you learn how computer scientists approach everyday decisions, and how to turn these ideas into Python code.

WHY IT MATTERS

More and more business decisions rely on data and algorithms. You don’t need to become a programmer, but understanding how algorithms think, and writing a few lines of code yourself, makes you a better decision maker. No prior knowledge required.

Roadmap

01 About this course: topics, format and how you pass

02 How to learn programming

03 Setting up Python

Six topics, one per session

- **I:** Optimal Stopping
- **II:** Explore & Exploit
- **III:** Caching
- **IV:** Scheduling
- **V:** Randomness
- **VI:** Computational Kindness

Take part, it pays off

- **Participate actively**: it makes the course easier and more fun
- Inspired by the book **Algorithms to Live By**[1]
- All material is online: courses.nilsroemer.com

1. Christian, B., & Griffiths, T. (2016). Algorithms to live by: the computer science of human decisions. First International edition. New York, Henry Holt and Company.

Speaker notes

Today’s agenda in detail: about the course and expectations, how to learn programming (mindset and resources), setting up Python with uv and VS Code, using notebooks with uv, Q&A and next steps.

Each session: lecture, then tutorial

- **Lecture:** we discuss an algorithm for everyday decisions
- **Tutorial:** step-by-step assignments, solved together in groups
- **Difficulty:** depends on your programming experience



No worries, we will help you out if you have any questions!

Speaker notes

Lecture: presentation and discussion of algorithms related to everyday decision-making. Tutorial: step-by-step assignments to be solved and discussed together in groups. Difficulty: strongly depends on your background and programming experience.

How you pass

- **Pass/fail course** without exams
- **75% attendance** required
- **Two assignments** on algorithmic problems from the course
- **Short final presentation** of your prototype in the last lecture
- You can work in groups of **up to three students**

How to hand in assignments

- Each group submits **one solution** per assignment
- Hand in your **working notebooks**
- Due by the **end of the quarter**, earlier is better
- At least **50%** of the solutions have to be correct

Speaker notes

Earlier submissions are highly encouraged: you get feedback while the topic is still fresh.

You learn Python in the tutorials

We will mostly **not cover Python during the lectures!**

Question: Does anybody know why?

- The best way to learn is by **doing**
- Lectures focus on the decision-making algorithms
- You learn Python by **doing the tutorials**

Speaker notes

In our experience, the best way to learn programming is by doing. Don’t worry, we will help you out if you have any questions.

Programming is like a new language

- At first it might be **a little bit overwhelming**
- First you get used to it and learn words
- Later you apply it and see results
- Important: *Practice, practice, practice!*

What you take home

- The **basics of programming**
- **Algorithmic thinking**: how computer scientists approach a problem
- Methods you can **apply** to practical problems

Why Python?

- **Simple syntax**: easy to read and to learn
- **Versatile**: web, data analysis, artificial intelligence
- **Big community**: help and documentation everywhere

Speaker notes

Origins: Python was conceived in the late 1980s as a teaching and scripting language. Its syntax is mostly straightforward and very easy to learn. It is used in web development, data analysis, artificial intelligence and more, and it has a large community of users worldwide with extensive documentation.

Learn first, then use AI

- In the tutorials, **write the code yourself**: that's how you learn
- At the end of the course, you build a small project **with AI support**
- Until then: if you use AI, ask it to **explain**, not to solve

Speaker notes

AI assistants are powerful, but learning to program means struggling a bit yourself. We use AI deliberately in the final project phase.

Roadmap

01 About this course: topics, format and how you pass

02 **How to learn programming**

03 Setting up Python

Three habits that work

1. **Be present**: attend the lecture and solve the tutorials
2. **Put in some work**: repeat code and try to understand it
3. **Do coding**: run code, play around, modify, and solve

Speaker notes

Great resources to start are books (see the list at the end of the lecture) and small challenges, for example on Codewars.

Don’t give up!

- Programming is **problem solving**, errors are normal
- Expect to **stretch** your comfort zone



Roadmap

01 About this course: topics, format and how you pass

02 How to learn programming

03 **Setting up Python**

VS Code is where you write code

- Like **Word for text**, but for code: an app to **write, run and find mistakes** in code
- Apps like this are called **IDEs** (Integrated Development Environments)
- We use VS Code for the **whole course**

Speaker notes

Other IDEs exist, for example PyCharm or Zed. We use VS Code because it is free, works on all systems and has good notebook support. Every step of the setup is in the Installation Guide on the website, with a check at the end of each step.

The terminal: typing instead of clicking

- A window where you **type a command**, press Enter, and the computer **answers with text**
- No menus, no buttons: the computer does exactly what you typed
- We need it only for the **setup**: a handful of commands, all in the guide to copy

Speaker notes

The terminal lives inside VS Code, as a panel at the bottom. On a Mac the line ends with a percent sign, on Windows it starts with PS for PowerShell. Nobody needs to memorise commands: during the setup, you copy them from the guide, paste them and wait until the line is back.

`uv` installs Python for us

- `uv` is a small **helper tool**: it installs Python and everything Python needs
- Everyone gets the **same Python version**, so code works the same on every laptop
- You install `uv` **once**, then it does the rest for you

Speaker notes

Without a tool like uv, every laptop ends up with a slightly different Python, and we would spend the tutorials fixing setups instead of programming. uv also keeps each project in its own folder with its own packages, so nothing gets mixed up.

Notebooks mix text and code

- A notebook is a document with **text cells** and **code cells**
- You run a code cell, and the result appears **right below it**
- All tutorials are notebooks: **read, run, change, try again**

Speaker notes

Notebooks are ideal for learning: you see a short explanation, run a few lines, and immediately see what happens. VS Code opens them with the Jupyter extension. The tutorials on the website can be downloaded as notebooks under “Other Formats”.

Now we install together

Every step, every command, every click is in the guide:

courses.nilsroemer.com/general/uv.html

- Follow the steps **in order**, don't skip any
- After each step: a small **check**
- We stop at four **checkpoints** and compare

Speaker notes

Open the guide now on your laptop, or scan the QR code with your phone. We go through the guide step by step and stop at the checkpoints. If you get stuck, raise your hand, and we help you before moving on.

Checkpoint 1: VS Code is open with both extensions

- VS Code shows its **welcome page**
- Under **Extensions**, both **Python** and **Jupyter** say “Installed”

GUIDE: STEPS 1 AND 2

Checkpoint 2: The terminal shows a version number for `uv`

- Your folder `programming-course` is open in VS Code
- A terminal is open at the bottom, and it answers to `uv` with a line like `uv 0.11.26`

GUIDE: STEPS 3 TO 5

Speaker notes

Most common problem here: uv is “not found”. Almost always, VS Code was not closed and reopened after the installation.

Checkpoint 3: You see Hello from programming-course!

- Python 3.13 is installed
- Your folder is a Python project, and the example program prints `Hello from programming-course!`

GUIDE: STEPS 6 AND 7

Checkpoint 4: Your first notebook shows Hello

- The notebook uses the **.venv** kernel with **Python 3.13**
- Below the cell with `print("Hello")`, you see `Hello`

GUIDE: STEPS 8 AND 9



That was the hardest part today! Step 10 of the guide: open a tutorial as a notebook.

Speaker notes

If the .venv kernel does not show up, reloading the VS Code window usually fixes it; the “Something doesn’t work?” table in the guide has the details. Whoever is done can already download the first tutorial, Step 10 in the guide.

What we covered

ABOUT THIS COURSE: TOPICS, FORMAT AND HOW YOU PASS

Six topics, each with a lecture and a Python session. You pass with 75% attendance, two assignments and a final presentation.

HOW TO LEARN PROGRAMMING

Programming is learned by doing: small steps, regular practice, and don’t give up.

SETTING UP PYTHON

Your laptop is ready: VS Code, Python installed with uv, and notebooks. Every step is in the Installation Guide.

Why it matters

More and more business decisions rely on data and algorithms. You don’t need to become a programmer, but understanding how algorithms think, and writing a few lines of code yourself, makes you a better decision maker.

Learn by doing.

Any questions so far?

After the break: Optimal Stopping

- The “Secretary Problem” and the 37% rule
- When to stop searching and make a decision
- Testing the rule ourselves with a small simulation

> **That’s it for our introduction!**
> Let’s have a short break and then continue with our first topic.

Literature

Interesting literature to start

- Christian, B., & Griffiths, T. (2016). Algorithms to live by: the computer science of human decisions. First international edition. New York, Henry Holt and Company.[1]

1. The main inspiration for this lecture. We have read it and discussed it in depth, always wanting to translate it into a course.

Books on Programming

- Downey, A. B. (2024). Think Python: How to think like a computer scientist (Third edition). O'Reilly. Free online
- Elter, S. (2021). Schrödinger programmiert Python: Das etwas andere Fachbuch (1. Auflage). Rheinwerk Verlag.



Think Python is free online and a great start. Schrödinger programmiert Python is a playful alternative in German.

More Literature

For more interesting literature, take a look at the literature list of this course.